FATEC RIBEIRÃO PRETO

Tecnologia em Análise e Desenvolvimento de Sistemas | 3º semestre
Estrutura de Dados | Prof. Me. Heber Castro

RELATÓRIO TÉCNICO

Sistema de gerenciamento de biblioteca com BST, grafo e estruturas lineares em C (Tema C — Biblioteca)

Débora Camila Santos | 2840482413042

https://github.com/devdeborasantos/projeto-estrutura-dados

Ribeirão Preto – SP

Novembro de 2026

Sumário

1. Introdução e objetivo 3
2. Modelagem e campos 3
2.1 Registro Livro 3
2.2 Arquitetura modular 3
2.3 Leitura segura de entradas 4
3. Justificativa das estruturas 4
4. BST e casos de remoção 5
4.1 Representação, inserção e busca 5
4.2 Remoção: os três casos 5
5. Grafo e BFS 6
5.1 Representação e mapeamento ID → índice 6
5.2 Arestas e regras de rejeição 6
5.3 BFS e consulta de caminho 6
6. Integração: lista, fila, pilha e ordenação 7
6.1 Consistência entre BST e grafo (rollback) 7
6.2 Pilha de histórico 8
6.3 Ordenação sobre vetor auxiliar 8
7. Análise de complexidade 8
7.1 BST e dependência da altura 9
7.2 Grafo e BFS 9
7.3 Ordenação 10
7.4 Pilha, fila e lista 10
8. Testes e resultados 10
9. Conclusão e limitações 11
10. Declaração de uso de IA 11
Referências 12

1. Introdução e objetivo

Este relatório documenta o projeto prático individual da disciplina de Estrutura de Dados: uma aplicação de terminal em linguagem C que resolve um problema prático integrando TADs, ponteiros, lista encadeada, pilha, fila, um algoritmo de ordenação, uma árvore binária de busca (BST) e um grafo. A BST e o grafo são os componentes centrais da avaliação.

Entre os três temas propostos (Redes, Rotas urbanas e Biblioteca), foi escolhido o **Tema C — Biblioteca**. Os registros são livros (ID, título, autor e ano de publicação) e as conexões entre livros (por exemplo, mesma série, mesmo autor ou leitura recomendada) são arestas de um grafo não direcionado e não ponderado. A BST indexa os livros pelo ID; o grafo relaciona os livros também pelo ID, nunca por ponteiros para nós da BST.

2. Modelagem e campos

2.1 Registro Livro

Cada livro é representado por uma `struct` com os campos da Tabela 1. Os textos são vetores de 80 posições (79 caracteres úteis mais o terminador). A estrutura ocupa 168 bytes e é armazenada **por valor** dentro do nó da BST, o que evita ponteiros compartilhados e simplifica a cópia para o vetor auxiliar de ordenação.

Tabela 1: Campos do registro Livro e validações

Campo	Tipo	Restrição e validação
`id`	`int`	Inteiro positivo e único. Rejeitados: zero, negativos, texto não numérico, overflow e ID já existente.
`titulo`	`char[80]`	1 a 79 caracteres; aceita espaços; deve conter ao menos um caractere não branco.
`autor`	`char[80]`	Mesmas regras do título.
`ano`	`int`	Inteiro entre 1 e 2026.

2.2 Arquitetura modular

O projeto é dividido em TADs com interface (`include/*.h`) e implementação (`src/*.c`) separadas. A Figura 1 mostra as dependências: `main.c` apenas faz a interface com o usuário; `entrada` valida as leituras; `bst`, `grafo`, `pilha` e `ordenacao` são TADs independentes; o módulo `sistema` concentra as operações compostas que envolvem BST, grafo e histórico; e o `grafo` usa a `fila` e a `lista` na BFS.

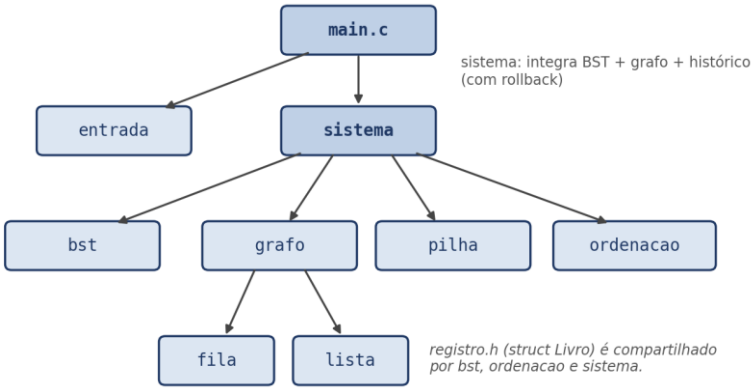


Figura 1: Dependências entre os módulos.

2.3 Leitura segura de entradas

Todas as leituras passam pelo módulo `entrada`. Cada linha é lida com `fgets` em um buffer de 256 bytes. Se a linha não termina em quebra de linha (isto é, excede o buffer), o restante é descartado com `fgetc` até o fim da linha e a entrada é rejeitada, de modo que o excesso não contamina a leitura seguinte. Textos com mais de 79 caracteres são rejeitados; textos vazios ou apenas com espaços também; espaços nas extremidades são removidos.

Os inteiros são convertidos com `strtol`, verificando `endptr` (lixo após o número), `errno == ERANGE` e os limites de `int` (overflow) e, por fim, o intervalo exigido (por exemplo, 1 a 2026 para o ano). Cada falha tem um código próprio (`ENT_VAZIO`, `ENT_LONGO`, `ENT_INVALIDO`, `ENT_OVERFLOW`, `ENT_FORA_INTERVALO`) e o `main` repete a pergunta até receber um valor válido. As funções recebem um `FILE*`, o que permite testá-las com arquivos temporários, sem digitação.

3. Justificativa das estruturas

A Tabela 2 resume porque cada estrutura foi escolhida e qual alternativa foi considerada. A escolha da matriz de adjacência e do Merge Sort foi condicionada ao tamanho do problema (no máximo 100 registros) e à necessidade de resultados determinísticos.

Tabela 2: Estruturas, uso e justificativa

Estrutura	Uso no projeto	Justificativa e alternativa descartada
BST	Índice dos livros por ID	Busca, inserção e remoção em O(h); a listagem em ordem de ID sai do percurso em-ordem, sem ordenar; menor/maior ID por descida simples. Vetor ordenado teria inserção O(n); lista, busca O(n); tabela hash não mantém ordem.
Grafo (matriz)	Conexões entre livros	Com V ≤ 100, a matriz custa 10 KB e testa uma aresta em O(1), simplificando duplicata, remoção e simetria. Lista de adjacência economizaria memória em grafos esparsos, mas exigiria manter os vizinhos ordenados por ID.

Estrutura	Uso no projeto	Justificativa e alternativa descartada
Mapeamento ID → índice	Vetores `id[]` e `ativo[]` do grafo	IDs são arbitrários (ex.: 5000) e não podem indexar a matriz. Cada vértice ocupa um slot compacto; slots de vértices removidos são reaproveitados.
Fila encadeada	BFS	A ordem FIFO garante a visita por níveis; fila encadeada não tem limite fixo.
Lista encadeada com ponteiro para o fim	Ordem de descoberta da BFS	Inserção no fim em O(1), preservando a ordem em que os vértices são descobertos.
Pilha encadeada	Histórico de alterações	Acesso LIFO à ação mais recente; limitada apenas pela memória. A consulta das cinco últimas percorre a pilha sem remover.
Merge Sort	Ordenação por título	O(n log n) em todos os casos, sem pior caso quadrático, e estável. Bubble, Selection e Insertion são O(n²) no pior caso; Quick Sort tem pior caso O(n²).

4. BST e casos de remoção

4.1 Representação, inserção e busca

Cada nó (`NoBst`) guarda uma cópia do `Livro` e dois ponteiros (184 bytes em plataforma de 64 bits). A árvore (`Bst`) guarda a raiz e a quantidade de nós. Como o ID é único, a BST nunca contém duplicatas: `bst_inserir` retorna `BST_DUPLICADO` sem alocar nada, e retorna `BST_SEM_MEMORIA` se o `malloc` falhar, sem alterar a árvore.

A inserção e a remoção são iterativas e usam um ponteiro para ponteiro (`NoBst **p`) que percorre a árvore apontando para o campo que referencia o nó atual. Assim, a raiz deixa de ser um caso especial: basta atribuir a `*p`. A busca desce comparando o ID; `bst_menor` e `bst_maior` descem sempre pela esquerda ou pela direita; `bst_em_ordem` é recursivo e recebe uma função de visita; `bst_copiar` usa essa mesma visita para copiar os registros, em ordem de ID, para um vetor de valores.

4.2 Remoção: os três casos

A remoção localiza o nó com o ponteiro `p` e trata os casos pelo número de filhos:

- **Folha (0 filhos):** o ponteiro do pai passa a ser o filho direito do nó, que é nulo, e o nó é liberado.
- **Um filho:** o ponteiro do pai passa a apontar para o único filho (esquerdo ou direito) e o nó é liberado.
- **Dois filhos:** localiza-se o sucessor em-ordem (o menor nó da subárvore direita), que é destacado de sua posição (seu filho direito ocupa o lugar dele) e religado no lugar do nó removido, assumindo seus dois filhos. O nó removido é liberado.

No caso de dois filhos, o sucessor é **religado**, e não tem seus dados copiados para o nó removido. A Figura 2 ilustra a remoção da raiz 50 da árvore usada nos testes: o sucessor

é o 60, que assume a raiz; o 70 fica com apenas o filho 80. A implementação também funciona quando o sucessor é o próprio filho direito do nó removido, sem tratamento especial.

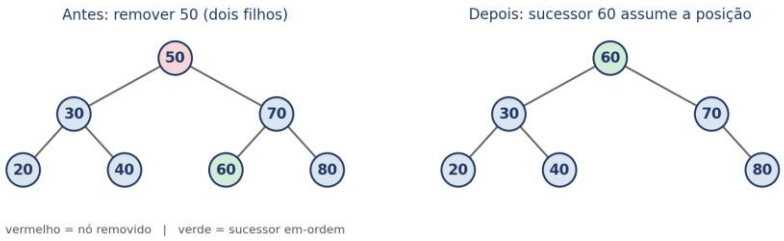


Figura 2: Remoção de um nó com dois filhos.

5. Grafo e BFS

5.1 Representação e mapeamento ID → índice

O grafo é não direcionado e não ponderado, implementado com matriz de adjacência `char adj[100][100]`. Como os IDs são arbitrários, a estrutura mantém dois vetores auxiliares: `id[i]` com o ID do vértice no slot `i` e `ativo[i]` indicando se o slot está em uso. Localizar um vértice é uma busca linear nos 100 slots, O(V). A criação ocupa o primeiro slot livre e zera sua linha e coluna; a remoção zera linha e coluna (removendo todas as arestas incidentes) e libera o slot para reutilização. O vértice recriado com o mesmo ID, portanto, não herda arestas antigas.

A capacidade é fixa em 100 vértices (`GRAFO_MAX`). Com o grafo cheio, a criação do vértice retorna `GR_CHEIO`, e a camada de integração desfaz a inserção já feita na BST (seção 6.1), de modo que o 101º cadastro é rejeitado sem deixar registro parcial.

5.2 Arestas e regras de rejeição

A aresta (A, B) é a mesma que (B, A): `adj[i][j]` e `adj[j][i]` são gravados juntos. As verificações são feitas nesta ordem, sempre antes de alterar o estado: existência dos dois IDs (`GR_VERTICE_INEXISTENTE`), laço A = A (`GR_LACO`) e aresta duplicada em qualquer ordem (`GR_ARESTA_DUPLICADA`). Na remoção, uma aresta inexistente retorna `GR_ARESTA_INEXISTENTE`. Em todos os casos o menu exibe uma mensagem e o grafo permanece inalterado.

5.3 BFS e consulta de caminho

A BFS recebe uma origem existente e devolve, em uma lista encadeada, todos os vértices alcançáveis, incluindo a origem, na ordem de descoberta. O vértice é marcado como visitado **ao entrar na fila**, o que impede que seja enfileirado duas vezes. Para que os vizinhos

sejam percorridos em ordem crescente de ID independentemente do slot que ocupam, a BFS calcula uma vez o vetor `ordem[]` com os índices dos vértices ativos ordenados por ID e, para cada vértice retirado da fila, varre `ordem[]` testando a matriz.

Uma origem isolada produz apenas o próprio ID. A Figura 3 mostra o grafo usado nos testes: a partir do 10, os vizinhos 1, 5 e 20 são visitados em ordem crescente, e depois o 7 (vizinho do 5), resultando em 10, 1, 5, 20, 7. Os vértices 50, 51 e 99 pertencem a outros componentes e não são alcançados.

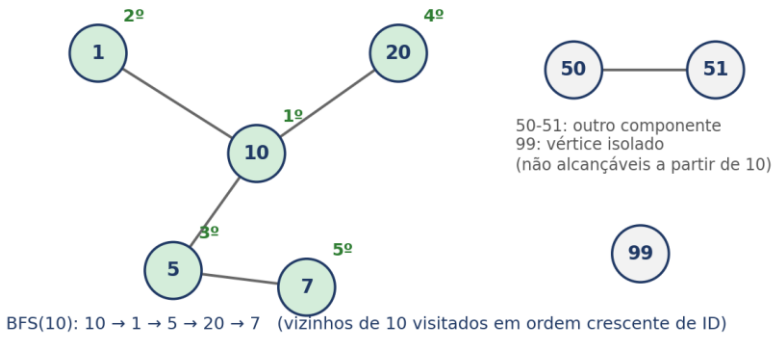


Figura 3: Grafo de exemplo e ordem de descoberta da BFS a partir do vértice 10.

A consulta de caminho executa a BFS a partir do primeiro ID e verifica se o segundo aparece na lista; não calcula o menor caminho. Se qualquer ID não existir, o sistema informa que o ID é inexistente, em vez de devolver um falso resultado de conectividade.

6. Integração: lista, fila, pilha e ordenação

6.1 Consistência entre BST e grafo (rollback)

O módulo `sistema` mantém a invariante **"o conjunto de IDs da BST é igual ao conjunto de vértices do grafo"** ao final de toda operação, inclusive nas que falham. Não existe operação independente para criar vértices: o vértice nasce do cadastro. A Tabela 3 resume a estratégia de rollback.

Tabela 3: Etapas das operações compostas e ação de desfazer

Operação	Etapa	Falha possível	Tratamento
Cadastro	1. Validar campos	ID ≤ 0, texto inválido, ano fora de 1 a 2026	Nada foi alterado.
Cadastro	2. Inserir na BST	ID duplicado; falha de `malloc`	Nada foi alterado.
Cadastro	3. Criar vértice	Grafo cheio (101º); falha de alocação	Remove da BST o ID recém-inserido.
Cadastro	4. Empilhar no histórico	Falha de `malloc`	Remove o vértice e o registro da BST.
Remoção	1. Buscar na BST	ID inexistente	Rejeita com mensagem, sem alterar.

Operação	Etapa	Falha possível	Tratamento
Remoção	2. Salvar vizinhos e remover vértice	—	Vizinhos guardados para eventual restauração.
Remoção	3. Remover da BST	Falha inesperada	Recria o vértice e restaura as arestas salvas.
Remoção	4. Empilhar no histórico	Falha de `malloc`	Aviso; BST e grafo permanecem consistentes.
Conexões	Adicionar/remover aresta	Laço, duplicada, inexistente	Validações precedem a alteração.

A ordem do cadastro (BST primeiro, depois vértice) permite detectar o ID duplicado antes de tocar o grafo e faz da rejeição por capacidade um caso de rollback bem definido. Para demonstrar os rollbacks sem depender de falhas reais de memória, a estrutura `Sistema` tem dois ganchos de simulação (`simular_falha_vertice` e `simular_falha_remocao_bst`) que provocam uma falha única e são usados apenas nos testes.

6.2 Pilha de histórico

A pilha é dinâmica e encadeada. Cada item guarda o tipo da ação (cadastro, remoção, adição ou remoção de conexão) e um ou dois IDs. Só entram no histórico as operações de alteração de estado **concluídas com sucesso**; buscas, listagens, ordenações, consultas e operações rejeitadas não são registradas. A pilha oferece `push`, `pop`, `peek` e uma consulta não destrutiva (`pilha_ultimas` e `pilha_imprimir_ultimas`) que percorre a pilha, sem remover nada, e exibe até cinco ações da mais recente para a mais antiga; com menos de cinco mostra as disponíveis e, se vazia, informa.

6.3 Ordenação sobre vetor auxiliar

A cada solicitação do menu, `sistema_ordenar` aloca um vetor, copia para ele os registros da BST **por valor** (`bst_copiar`), ordena o vetor com `ordenar_livros`, exibe o resultado e libera o vetor. A BST não é alterada. O critério é o título, comparado com `strcmp`: a comparação é sensível a maiúsculas e minúsculas e não normaliza acentos. Se dois títulos forem idênticos, desempata-se pelo ID crescente. Como o ID é único, a ordem resultante é total e determinística.

O algoritmo implementado é o Merge Sort próprio (sem `qsort`). Se o `malloc` do buffer auxiliar do Merge Sort falhar, a função recorre a um Insertion Sort, que não precisa de memória extra. A BFS usa a fila e a lista como descrito na seção 5.3.

7. Análise de complexidade

7.1 BST e dependência da altura

O custo de busca, inserção e remoção na BST depende da altura h da árvore, e não diretamente do número de nós n. Em uma árvore balanceada, $h \approx \log_2 n$ e as operações custam O(log n). Porém, a BST implementada não se autobalanceia: se os IDs forem cadastrados em ordem crescente (ou decrescente), cada nó tem apenas um filho e a árvore **degenera** em uma lista encadeada, com h = n e custo O(n) por operação. O teste de degeneração do projeto confirma isso: 10 IDs crescentes produzem altura 10.

No sistema, o número de registros é limitado a 100 pela capacidade do grafo. Logo, uma árvore com 100 nós tem altura entre 7 (melhor caso, completa) e 100 (pior caso, degenerada), e o pior caso custa no máximo 100 comparações por operação. A recursão do percurso em-ordem, da altura e da liberação tem profundidade h ≤ 100, sem risco de estouro de pilha. Em um sistema sem esse limite, seria recomendável uma árvore balanceada (AVL ou rubro-negra).

Tabela 4: Complexidade temporal das operações da BST

Operação	Melhor	Médio	Pior	Observação
Inserir / buscar	O(1)	O(log n)	O(n)	Melhor caso: alvo na raiz. Pior: árvore degenerada.
Remover	O(1)	O(log n)	O(n)	Busca do nó + localização do sucessor (no máximo h passos).
Menor / maior	O(1)	O(log n)	O(n)	Desce uma aresta por nível.
Em-ordem / cópia	O(n)	O(n)	O(n)	Cada nó é visitado uma vez; pilha de recursão O(h).
Altura / liberar	O(n)	O(n)	O(n)	Percorrem todos os nós.

Em espaço, a BST usa O(n): 184 bytes por nó. As operações de inserção, busca e remoção são iterativas e usam O(1) de memória adicional.

7.2 Grafo e BFS

A matriz de adjacência ocupa espaço fixo $O(V^2)$: o `sizeof(Grafo)` é de 10.504 bytes. A BFS visita cada vértice alcançável uma vez e, para cada um, varre o vetor `ordem[]` (até V posições) consultando a matriz em O(1); o custo é, portanto, $O(V^2)$, **independente do número de arestas**. O cálculo de `ordem[]` por inserção custa $O(V^2)$ no pior caso, mantendo o total em $O(V^2)$. Com uma lista de adjacência o custo seria O(V + E), mas seria preciso manter ou ordenar os vizinhos para cumprir a ordem crescente de ID. Com V ≤ 100, a BFS faz no máximo da ordem de 10^4 operações elementares.

Tabela 5: Complexidade das operações do grafo (matriz de adjacência)

Operação	Tempo	Espaço extra
Criar / remover vértice	O(V) — busca do ID e do slot; limpeza de linha e coluna	O(1)
Adicionar / remover aresta; conexão direta	O(V) para localizar os índices; O(1) para a matriz	O(1)
Vizinhos em ordem crescente de ID	$O(V^2)$ no pior caso (ordenação de índices por inserção)	O(V)
BFS (alcançáveis)	$O(V^2)$	O(V): `visitado`, fila, `ordem[]` e lista
Existência de caminho	$O(V^2)$: BFS + busca do destino na lista, O(V)	O(V)

7.3 Ordenação

Foi adotado o Merge Sort porque seu custo é O(n log n) nos três casos (melhor, médio e pior), e porque é estável. Seu preço é O(n) de memória auxiliar. O Merge Sort executa entre $(n/2)\cdot\log_2 n$ e $n\cdot\log_2 n - n + 1$ comparações. Cada comparação por `strcmp` custa O(L), com $L \leq 79$; como L é limitado, o custo é tratado como constante.

Cada solicitação de ordenação custa O(n) para copiar a BST mais O(n log n) para ordenar, com O(n) de memória temporária: para n = 100, cerca de 16,8 KB para o vetor de cópias e outro tanto para o buffer do Merge Sort, ambos liberados ao final. O fallback com Insertion Sort, usado apenas se o buffer não puder ser alocado, custa O(n) no melhor caso e $O(n^2)$ no pior.

7.4 Pilha, fila e lista

`push`, `pop` e `peek` da pilha, `enfileirar` e `desenfileirar` da fila e a inserção no fim da lista (graças ao ponteiro para o último nó) são O(1) em tempo e espaço. A consulta das cinco últimas ações percorre no máximo cinco nós, ou seja, O(1). O espaço total de cada estrutura é O(n) no número de elementos armazenados (24 bytes por item de histórico).

8. Testes e resultados

Os testes estão em `tests/test_main.c` e não usam framework. Cada verificação imprime a área, o cenário, o resultado esperado e o resultado obtido; o programa retorna código diferente de zero se houver falha. As entradas de texto e números são simuladas com arquivos temporários, o que torna a suíte totalmente reproduzível. Na execução de `make test` [compilação com `-std=c11 -Wall -Wextra -Wpedantic`, versão 14.2.0 (Rev2, Built by MSYS2 project) em Windows 11] foram executadas 120 verificações, todas com resultado obtido igual ao esperado e **0 falhas**. Um trecho da saída real do programa de testes é mostrado a seguir.

Além dos testes automatizados, a aplicação foi usada pelo menu em uma sessão manual: após cadastrar três livros e conectar 10 e 20, repetir a conexão exibiu “Conexao ja existe.”, ligar 10 a si mesmo exibiu “Laco (A,A) nao permitido.”, a BFS a partir do 10 listou 10 → 20 e a consulta de caminho entre 10 e 30 respondeu que não existe caminho.

Como reproduzir: `make test` executa a suíte; `make` e `./bin/biblioteca` abrem o menu.

9. Conclusão e limitações

O projeto atende aos requisitos: BST completa com os três casos de remoção, grafo com mapeamento ID → índice, capacidade de 100 vértices, BFS com fila e lista, pilha de histórico com consulta não destrutiva, ordenação própria sobre vetor auxiliar, entradas validadas e integração com rollback. A separação em TADs com interface e implementação distintas facilitou testar cada parte isoladamente nas etapas do desenvolvimento, e a estratégia de manter a invariante entre BST e grafo simplificou o raciocínio sobre falhas.

Limitações conhecidas:

- A BST não é balanceada; no pior caso (IDs em ordem) o custo por operação é O(n), embora limitado a 100 pelo tamanho do sistema.
- A capacidade é fixa em 100 vértices; a matriz custa $O(V^2)$ de espaço mesmo com poucas arestas.
- `strcmp` compara bytes: não há normalização de acentos nem de maiúsculas e minúsculas, portanto, títulos acentuados podem aparecer fora da ordem alfabética esperada por um leitor.
- Se o registro no histórico falhar após uma remoção ou conexão já concluída, o sistema apenas emite um aviso; BST e grafo permanecem consistentes, mas a ação não aparece no histórico.
- As falhas de alocação são simuladas por ganchos de teste, não por injeção real de falhas de `malloc`.
- Não há persistência em arquivo nem operação de desfazer.

Como evolução, seriam naturais: uma árvore AVL, lista de adjacência dinâmica sem limite de vértices, persistência em arquivo, DFS, componentes conexos e o uso da pilha para desfazer operações.

10. Declaração de uso de IA

Foram utilizadas as ferramentas de inteligência artificial **Claude (Anthropic) e ChatGPT (OpenAI)** como apoio neste trabalho em alguns pontos onde surgiram dúvidas ou erros de execução.

Referências

CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. **Algoritmos: teoria e prática**. 3. ed. Rio de Janeiro: Elsevier, 2012.

KERNIGHAN, B. W.; RITCHIE, D. M. **The C Programming Language**. 2. ed. Englewood Cliffs: Prentice Hall, 1988.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC 9899:2011 - Programming languages - C**. Genebra: ISO, 2011.

CHACON, S.; STRAUB, B. **Pro Git**. 2. ed. Apress, 2014. Disponível em: https://git-scm.com/book.